

基于尺度自标定与形状先验校正的粘连米粒计数方法

数字图像处理课程作业

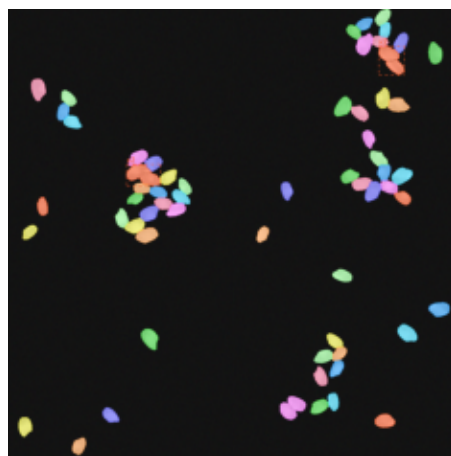
摘要：米粒挨在一起时，二值化后会连成一块、只被数作一粒，米越密漏得越多。本文让程序先从图像自身量出一粒米有多大、多长、多凸，再据此判断哪一块是几粒粘在一起、应当分成几粒。分完后按面积与形状复核，并剔除硬币、木框一类不是米的目标。判据中没有写死的像素数，换一批照片不必重调参数。在真实照片、自制粘连合成图与低分辨率图像三组数据上，平均每张分别差 4.84、0.25 与 1.90 粒，优于五种经典方法。粘连最重的一档，直接数连通域平均少数 38.9 粒，本文方法只差 0.9 粒。本文还以视觉大模型 SAM 3 的输出为参考答案，训练出一个 27 万参数的小网络，平均每张差 0.68 粒，而体积约为 SAM 3 的三千分之一。文中也记录了试过而未奏效的做法与尚未解决的问题。

关键词：米粒计数；粘连目标分割；标记控制注水分割；尺度自标定；知识蒸馏

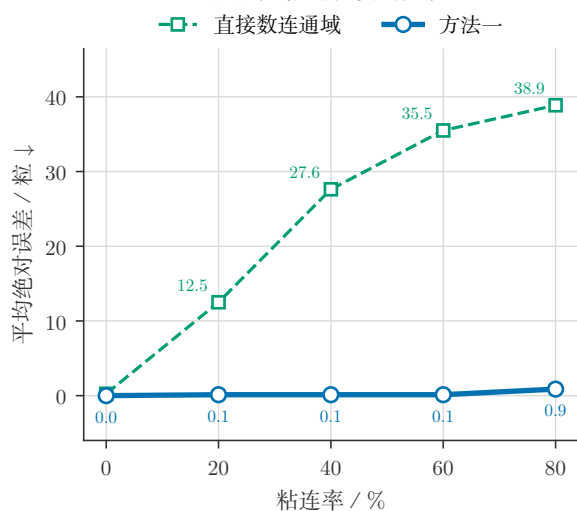
真实照片，数出 99 粒，真值 98 粒



合成图，粘连率 80%，数出 63 粒，真值 64 粒



误差随粘连程度的变化



四种做法在同一批测试图上的误差

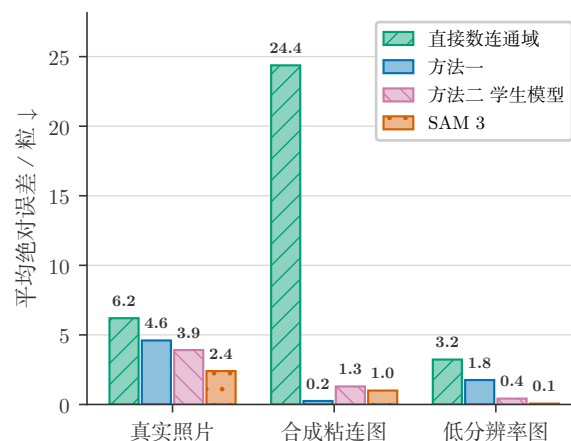


图 1: 上排为方法一在真实照片与重度粘连合成图上的逐粒计数结果；下排左为计数误差随粘连率的变化，右为四种做法在同一批测试图上的平均绝对误差。

目 录

1 引言	3
2 相关工作	3
3 方法一几何方法	4
3.1 几个基本概念	4
3.2 自适应二值化	5
3.3 米粒尺度自标定	6
3.4 判断是否粘连	6
3.5 放种子并切开粘连块	8
3.6 检查与修正	9
4 方法一的实验与分析	10
4.1 数据集	10
4.2 评价指标	10
4.3 与其他方法比较	10
4.4 误差随粘连程度的变化	11
4.5 图像质量变差时的表现	11
4.6 消融实验	13
4.7 与 SAM 3 对比	13
4.8 算力成本对比	15
4.9 本节小结	15
5 方法二 SAM3 作为教师模型训练专用小模型	16
5.1 方法二思路的由来	16
5.2 具体实现方式	16
5.3 结果与代价	17
6 讨论	17
6.1 让程序跑得更快	17
6.2 尝试过但效果不好的办法	18
6.3 一些不足	18
7 结论	19
A 更多计数结果	19
A.1 真实照片 D1	19
A.2 合成粘连图 D2	20
A.3 低分辨率图 D3	21
B 代码、数据与复现方式	22

1 引言

本次作业要求数出照片里有多少粒米，并解决连体米粒计数不准的问题。

恰好我大一大二打过两次电赛，负责视觉循迹，目标识别。于是我先利用已有的知识进行了初步尝试。

我先按最基本的思路做了一遍。先转灰度，再用 Otsu 法^[1]自动选一个阈值把米粒和背景分开，最后统计连通域的个数。连通域指二值图中连在一起的一片前景像素，正常情况下一粒米就是一个连通域。米粒互不接触时，这一流程没有问题。一旦两粒米挨在一起，它们在二值图中连成一块，只被计为 1 粒。米粒越密，漏计越多，见图 2。实测粘连率 80% 时平均少数 38.9 粒，这条误差曲线见图 12，也正是本次作业的难点所在。

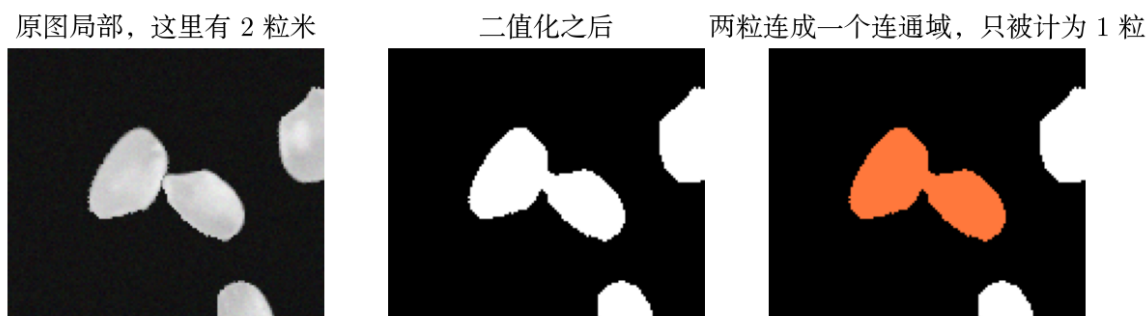


图 2: 粘连导致漏计的局部放大。右图中橙色的一整块是一个连通域，其中实际有 2 粒米。

于是我转向网上学习新方法，在查阅文献时注意到，这个问题在已发表的工作中也常被回避。Kurade 等人^[2]的米质检测系统采用“腐蚀取标记结合注水分割”，注水分割即 watershed 算法，常译作“分水岭算法”。该文承认这一算法分不开互相接触的米粒，因而把这些米粒从数据集中剔除。本文试着解决这一问题。

本文的工作可以概括为以下四点。

1. 由程序从图像自身量出单粒米的尺寸与形状，后续判据均以此为单位，像素常数是不固定的；
2. 不预先指定颜色通道与阈值方式，而是把若干组合逐一试过，按一个打分函数自动选取；
3. 以“又大又凹”为条件，判断连通域是否为粘连块，再用注水分割切开；
4. 切分后按面积与形状复核，并剔除硬币、边框等非米目标。

在三组性质不同的数据上，上述方法的平均绝对误差为 4.84、0.25 与 1.90 粒，优于本文实现的五种经典对比方法；粘连率 80% 时误差约为直接统计连通域的四十四分之一。在此之外，本文还用 SAM 3 的输出作伪标签蒸馏了一个 27 万参数的小网络，详见第 5 节，其总体误差进一步降到 0.68 粒。代码、数据来源与复现方式见附录 B。

2 相关工作

分开粘连目标大致有两种思路。

一种是从轮廓入手。两粒米的接触处在轮廓上是一个凹点，找到凹点再连线切开。Tan 等人^[3]用这种方法处理粘连稻谷，Avzalov 等人^[4]进一步用椭圆拟合滤除假凹点。本文将后者实现为对比方法 B5。

另一种是从区域入手，先用注水分割^[5]切开，再修正结果。Gamarra 等人^[6]先以大小和凸性判断哪些区域需要切分，切开后合并过碎的部分。本文方法属于后一类，结构上与之相近；我实现该方法以后查文献，才发现有前辈用过类似的方法的。

国内在谷物考种一类的任务上做过不少同类工作。凌云等人^[7]较早把注水分割用在谷物籽粒上，杨蜀秦等人^[8]处理的正是连接在一起的米粒。荀一等人^[9]在连续腐蚀与膨胀的过程中找出两粒之间的公共区域，取其端点连线切开。柳冠伊等人^[10]把距离变换图当作地形图，沿山谷线切分。他们还指出，这样一来就不必先由人工设定阈值去判断是否粘连。这一点与本文用自标度量出判据的想法相近。李文勇等人^[11]则先用形状因子判断一个区域是不是粘连，再定位分割点。判断的那一步与本文第 3.4 节做的是同一件事，只是所用的量不同。

3 方法一几何方法

这是本文的第一种方法，步骤是“先二值化，之后形态学，最后判断连通域”。不同之处在于把其中的参数交给图像自己决定。整体流程见图 3，左列是对整张图的处理，右列是对每个连通域的判断与计数，各步骤在一张合成图上的中间结果见图 4。

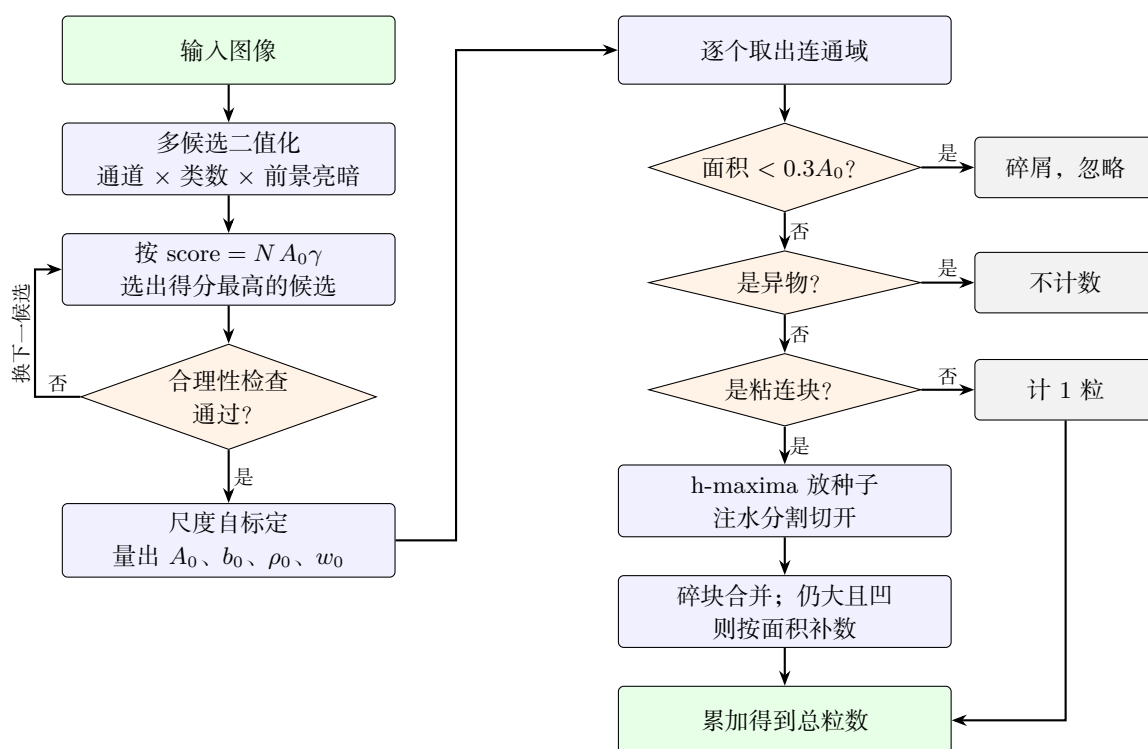


图 3: 方法流程图。

3.1 几个基本概念

距离变换 D 是指对每个像素，算出它到最近背景像素的距离。米粒中间的值大、边缘的值小，像一座山；两粒米粘连的地方会出现一道山谷。

种子 是人为指定的若干起点，一个种子代表一个要分出来的目标。放种子相当于告诉程序“这里有一粒米”，它并不需要标出米粒的完整轮廓，落在米粒内部的一小块像素即可。

注水分割 相当于把距离图看成地形图，从每个种子处开始“注水”。水面自种子向四周漫开，两股水相遇的地方就是分界线。给几个种子就切成几块，所以一个粘连块最后被算作几粒，完全取决于种子怎么放。图 5 把这张地形画了出来。距离变换取负值后，每粒米陷成一个盆地，两粒贴合处隆起一道埂，水从两个盆底的种子涨上来，在埂上相遇，那里就是切分线。

凸实度 ρ 是区域面积与其凸包面积之比，凸包即包住该区域的最小凸多边形。一粒米接近凸形， ρ 接近 1；两粒粘在一起，接缝处凹进去， ρ 会变小。

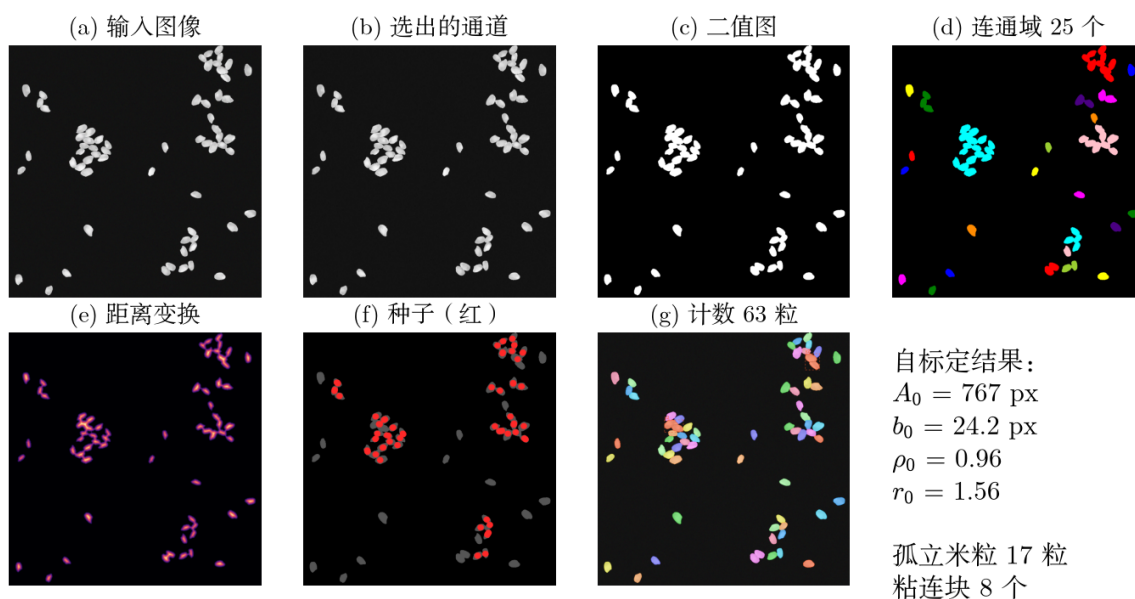


图 4: 各步骤在一张合成图上的中间结果。

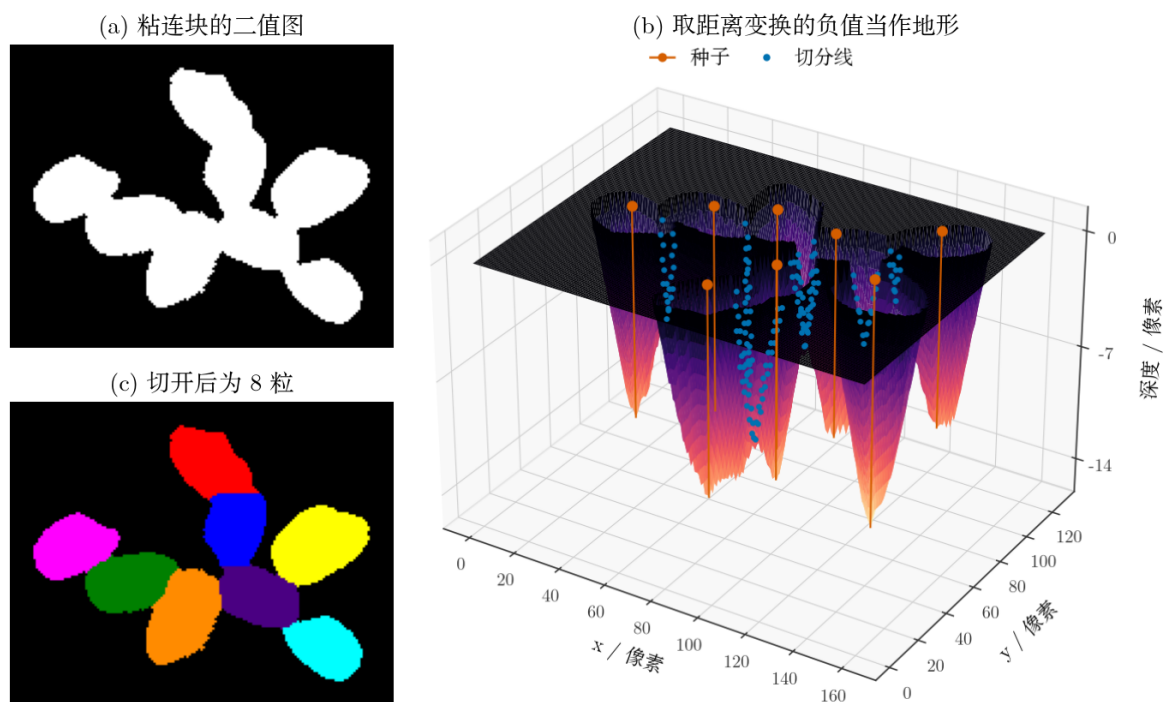


图 5: 把距离变换负值当作地形。(a) 为一个两粒粘连块, (b) 为它的三维地形, 竖轴为距背景的像素距离, (c) 为注水后的切分结果。

3.2 自适应二值化

起初直接用“根据灰度二值化”的方法,但在真实照片上测试时出了问题。白米放在深色衬布上,衬布又铺在浅色桌面上,画面有三个亮度层次。两类阈值只能切一刀,无论切在哪里,都会有一整片背景被划进前景。图 6 是同一张照片的两种取法,取亮的一类作前景时桌面连成一块,占画幅的 33%;取暗的一类时衬布连成一块,占 65%,整张图只剩 6 个连通域。这一块伪前景比所有米粒加起来还大,后面的标定与计数都会被它带偏。

由此可见,通道、类数与前景极性都不宜事先指定。本文把这三者展开成候选集合。通道取灰度或 HSV 饱和度,阈值取 2 类或 3 类 Otsu,前景取亮或暗。每种组合都二值化一遍,再用

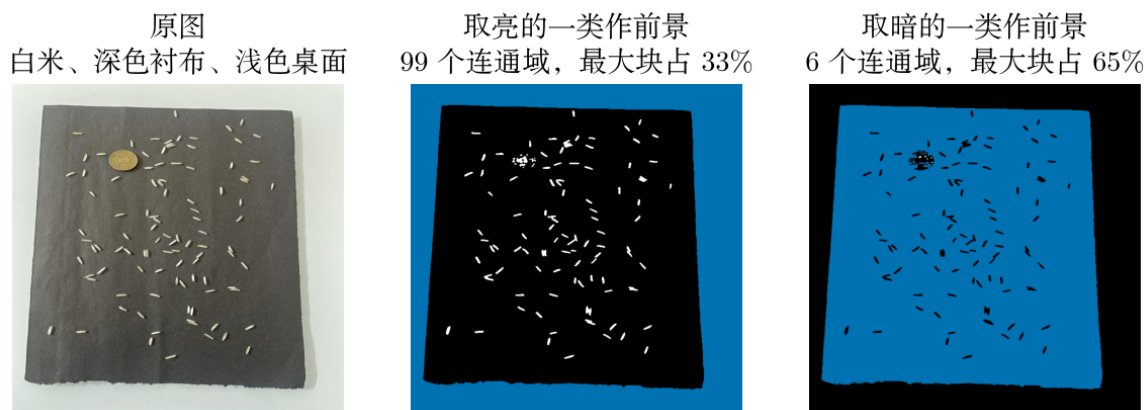


图 6: 固定用灰度两类阈值的后果, 蓝色为该二值图中最大的连通域。前景取亮或取暗都会多出一片巨大的伪前景。

下式打分择优。

$$\text{score} = N \cdot A_0 \cdot \gamma \quad (1)$$

式中 N 为尺寸接近单粒的连通域个数, A_0 为估计的单粒面积, γ 为前景与紧邻背景的灰度差。 γ 一项是后来补上的。合成图的饱和度通道会被噪声打成上千个小斑点, 只算前两项时它们反而得分最高。这些斑点与背景几乎没有反差, 计入 γ 后便被排除。

此外还有若干合理性检查, 例如米粒长宽比应在 1.25 至 8 之间、单粒面积不超过画幅的 1%。这些检查最初只设了下界。结果 D3 中有 67 张图把木框条纹当成了“米粒”, 其长宽比达 25, 一条就占画面的 3%。此后我补上了判断条件的上界, 并让检查不通过的候选回退到下一个。这 67 张图的平均误差由 11.45 粒降至 3.57 粒。

3.3 米粒尺度自标定

后续的多大算大、多凹算凹都需要一个参照物。三组数据的米粒尺寸相差悬殊, 短轴自 4 像素至 25 像素。固定的像素数难以通用, 只能由程序从每张图上把米粒尺度测出来。

最初的想法是取面积分布中最小的那个峰作为单粒, 实测常常选中比米粒更小的碎屑。改取出现次数最多的尺寸, 也就是面积分布的最高峰, 才稳定下来, 实现见代码 1。面积先取对数, 因为粘连块的面积大约是 A_0 、 $2A_0$ 、 $3A_0$ ……, 取对数后这些峰间距相等, 更容易看清, 见图 7。得到 A_0 后, 再从同一批接近单粒的连通域里量出短轴 b_0 、凸实度 ρ_0 、宽度 w_0 。

代码 1: 估计单粒面积 A_0

```
1 def estimate_a0(areas):
2     """取对数面积分布的最高峰作为单粒米面积"""
3     log_s = np.log(areas)
4     kde = gaussian_kde(log_s) # 核密度估计, 相当于平滑的直方图
5     grid = np.linspace(log_s.min(), log_s.max(), 512)
6     a0 = np.exp(grid[np.argmax(kde(grid))]) # 最高峰对应的面积
7     # 再取 0.65~1.45 倍 A0 之间的连通域求平均, 结果更稳
8     band = areas[(areas >= 0.65 * a0) & (areas <= 1.45 * a0)]
9     return band.mean() if len(band) > 0 else a0
```

3.4 判断是否粘连

连通域比单粒米大, 未必就是粘连, 也可能只是一粒偏大的米。两粒米贴合处一般会形成凹陷, 因此本文要求“既大且凹”两个条件同时成立。起初用的是“或”, 本想宁可多切, 实测在真实照片上误差反而增加 1.2 粒。

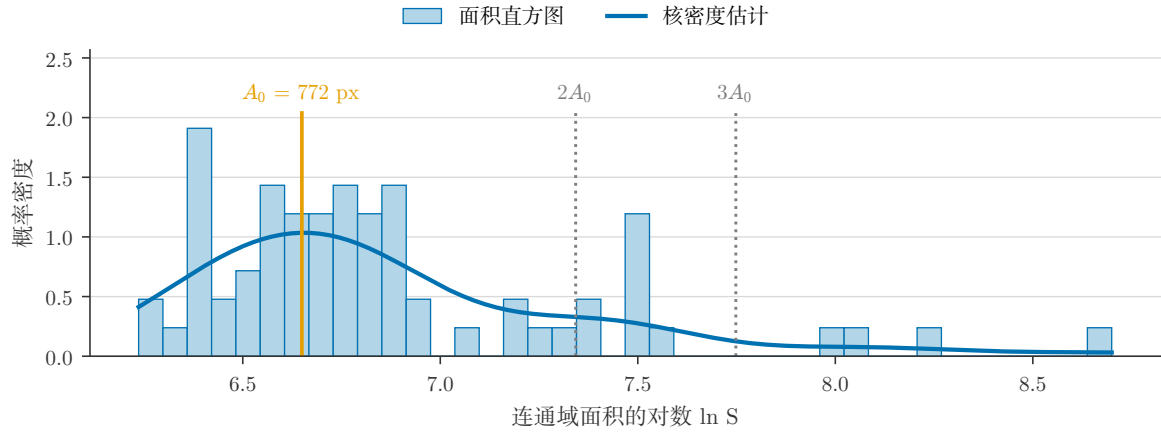


图 7: 面积分布的最高峰即单粒面积 A_0 , 粘连块集中在它的整数倍附近。

米粒很小时, 边缘锯齿明显, 凸实度已不可靠。这时改用区域内最大内切圆的直径 w , 它等于距离变换最大值的两倍。两粒米并排时面积翻倍而“厚度”不变, 所以改用面积与厚度的比值来判断, 实现见代码 2。一粒大米的这个值约为 1, 两粒并排约为 2。图 8 把两种情形并排画出。单粒与其凸包几乎重合, 凸实度 0.96。两粒粘连处留下明显的凹口, 凸实度降到 0.75。但两者的最大内切圆直径分别是 26 与 25 像素, 相差无几。

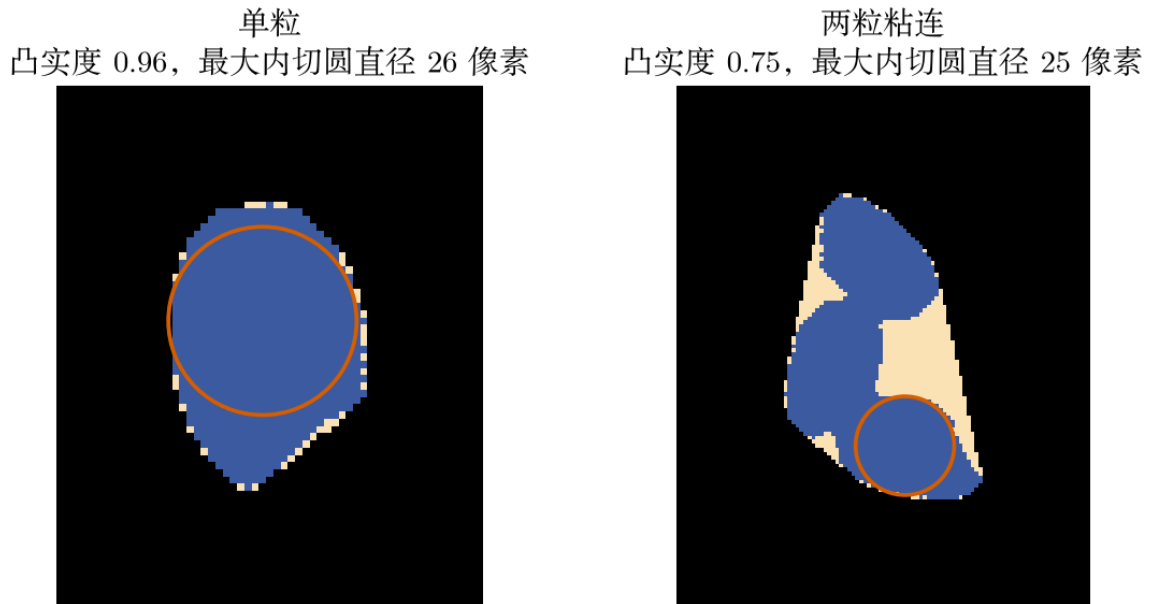


图 8: 单粒与粘连块的对照。蓝色为区域本身, 浅色为凸包多出的部分, 橙圈为最大内切圆。

代码 2: 判断一个连通域是不是粘连块

```
1 def is_touching(c, cal):
2     """c 是一个连通域, cal 是自标定得到的单粒米参数"""
3     if c.area <= 1.2 * cal.a0:                # 不比一粒米大, 不算粘连
4         return False
5     if cal.b0 < 4.5:                          # 米粒太小, 凸实度量不准, 改看厚度
6         e = (c.area / cal.a0) / (c.width / cal.w0) ** 2
7         return e > 1.5
8     return c.solidity < cal.rho0 - 0.06      # 又大又凹才算粘连
```

3.5 放种子并切开粘连块

判出粘连块之后，还要决定给注水分割放几个种子，也就是认定这一块里有几粒米。最先想到的是限制种子之间的最小距离，稍作计算便知此路不通。米粒细长，长宽比约 3.7。一粒米内部距离变换的“山脊”比两粒并排时的中心间距还长，最小距离取多少都难以兼顾这两个要求。

换一个角度就有了出路。两粒米之间是一道深谷，一粒米内部只是浅浅的起伏，该看的是深度而非距离。为此对距离变换取 h -maxima，把高度不足 h 的小峰抹平，实现见代码 3。 h 取 $0.1b_0/2$ ，但不低于 1 像素。用短轴 b_0 ，是因为两粒米能靠多近取决于它们的宽度。下限定为 1 像素，是因为距离变换本身就以像素计量，更小的起伏只是边缘的锯齿。图 9 沿块内的山脊从一个种子走到另一个种子，取了一条剖面。较低的峰高 12.2，谷底 4.2，谷深 8.0，超过 $h = 1.2$ ，因而被判为两粒。一粒米内部的起伏不到 h ，会被抹平，不会被切开。

这里还有一个坑，是在检查效果图时才发现的。 h -maxima 在浮点的距离图上，会把同一个平顶的峰碎成几小块，差别只在舍入误差上，每一块都会变成一颗种子，于是一粒米被切成两半。实测同一粒米上碎出来的种子相距最多 0.55 倍短轴，不同米粒的种子相距至少 0.67 倍，所以把相距不到 0.6 倍短轴的种子并成一颗。

这个坑和另一个问题是连在一起的。碎种子没合并时， h 调得越低，碎出的种子越多，一粒米被切成两半的情况也越多，调参就把 h 推到了 $0.45b_0/2$ 。这么高的门限又把饱满米粒之间那道浅颈抹平了，许多粘连块切不开，只能按面积补数。两个错误的数量碰巧能相互抵消，所以总数看起来对，效果图却明显不对。合并碎种子之后再把 h 降下来，D2 上被切成半粒的块由 61 个降为 0，没切开的块由 364 个降为 39，D2 的误差由 1.77 粒降到 0.25 粒。

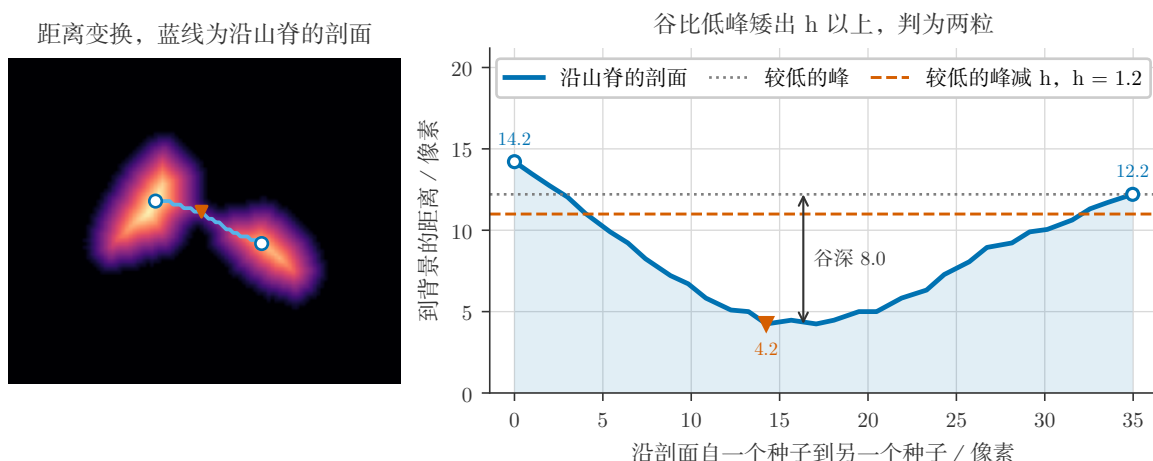


图 9: 为什么看深度而不看距离。左为粘连块的距离变换，右为沿山脊的剖面。谷底低于红色虚线，即比较低的峰矮出 h 以上，两个峰各留一个种子。

代码 3: 放种子并做注水分割

```
1 def split_cluster(mask, cal, beta=0.1):
2     dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
3     h = max(beta * cal.b0 / 2, 1.0)          # 深度门限跟着米粒宽度变，不低于 1 像素
4     peaks = h_maxima(dist, h)                # 只留下足够深的山峰
5     r = round(0.6 * cal.b0 / 2)             # 相距不到 0.6 倍短轴的峰算同一粒米
6     seeds = label(cv2.dilate(peaks, disk(r))) * peaks
7     if seeds.max() <= 1:                     # 只有一个种子，不用切
8         return (mask > 0).astype(int)
9     return watershed(-dist, seeds, mask=mask > 0)
```

3.6 检查与修正

注水分割只回答了“可能在哪里切”，切得对不对还要再检查。一个粘连块从切开到定出粒数的全过程见图 10，整个计数过程见代码 4。碎块并入邻居。切完后仍然又大又凹的块，说明没切开，就按面积补数。面积超过 $3A_0$ ，并且厚度满足 $w > 2.5w_0$ 或者和单粒一样凸的，判为异物。

代码 4: 计数主循环

```

1 total = 0
2 for c in components:
3     if c.area < 0.3 * a0:                # 碎屑，忽略
4         continue
5     if is_foreign(c, cal):                # 硬币、木框等，不计数
6         continue
7     if not is_touching(c, cal):          # 单独的一粒
8         total += 1
9         continue
10    parts = merge_small(split_cluster(c.mask, cal), a0) # 小于 0.4*A0 的碎块并入邻居
11    for p in regionprops(parts):
12        k = p.area / a0
13        if k < 0.4:                      # 没有邻居可并的碎块
14            continue
15        if k > 1.5 and p.solidity < cal.rho0 - 0.03:
16            total += round(k)             # 没切开，按面积补数
17        else:
18            total += 1

```

异物判据的依据有两条。米粒只是相互挨着，不会融为一体，因此一堆米最厚处仍只有一粒米那么厚。几粒米粘连通常还会留下凹口，面积数倍于单粒却与单粒一样凸的区域，多半不是米堆。实测单粒的 w/w_0 介于 0.98 与 1.14 之间，真实粘连块不超过 2.0，而硬币为 8.6、木框条约为 4。该判据最初是照着 D1 中的硬币写的，要求“更凸且更圆”；D3 的木框条比米粒更细长，恰好从“更圆”这一条漏了过去。去掉这一多余条件后，D3 误差由 2.34 粒降至 1.55 粒，D1、D2 则没有变化。

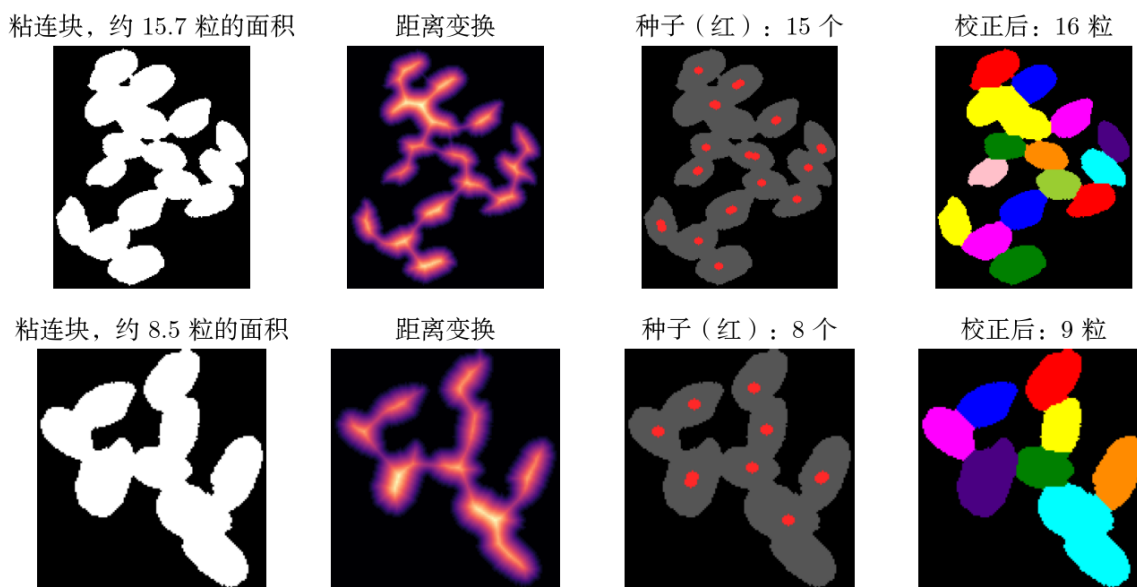


图 10: 粘连块的处理过程，自左至右为粘连块、距离变换、种子与最终结果。

4 方法一的实验与分析

这一节用实验回答五个问题。方法一比常见做法准多少，粘连加重时还准不准，图像变差时从哪里开始失效，每一步各起多大作用，以及和视觉大模型 SAM 3 相比差在哪里、省在哪里。

4.1 数据集

三组数据中，D1、D3 与 D2 所用的米粒素材取自 Roboflow Universe 上的公开数据集，许可为 CC BY 4.0。第 5 节的方法二沿用这三组数据。

D1, 真实照片, 51 张。取自 Roboflow 上的 rice 数据集^[12], 手机拍摄, 深色布上摊开的米粒, 每张 80–116 粒, 带一枚参照硬币, 每粒米都有人工标注框。米粒基本不粘连。

D2, 自制合成图, 40 张。从 RICE 数据集^[13]中 Karacadag 品种的单粒米照片里抠出米粒, 随机旋转、缩放后贴到深色画布上, 粘连率分 0%、20%、40%、60%、80% 五档, 每档 8 张, 每张 44–98 粒。它的真值由构造方式精确给定, 粘连程度也可控, 专门用来考粘连。

D3, 低分辨率, 719 张。取自 Roboflow 上的 RICE 数据集^[14], 224×224 的小图, 每张约 14 粒。该数据集导出时做过翻转增强, 同一源图存在多个副本, 按源图名去重后保留 719 张。

4.2 评价指标

记第 k 张图的真值为 g_k 、预测为 p_k 。主要指标为平均绝对误差 $MAE = \frac{1}{n} \sum_k |p_k - g_k|$, 即平均每张数错几粒。另外给出准确率 $1 - MAPE$ 与最差单张误差, 其中 MAPE 为平均相对误差 $\frac{1}{n} \sum_k |p_k - g_k| / g_k$ 。平均值小并不排除个别图像错得离谱, 所以两者一并看。

4.3 与其他方法比较

先看方法一比常见做法准多少。为使比较只落在“如何分开粘连”这一步上, 所有方法都跑在同一份二值图上。B1 直接统计连通域个数, 不对粘连作任何处理。B2 用前景总面积除以单粒面积。B3 是距离变换注水分割, 种子取 $D > 0.5 \max D$, 这是 OpenCV 教程的做法。B4 腐蚀后取连通域作种子, 即文献^[2]的做法。B5 为轮廓凹点 + 椭圆拟合^[4]。

结果见表 1。三组数据上, 方法一的平均误差都是六种几何方法中最低的。三种对比方法的失效值得单独说一下。

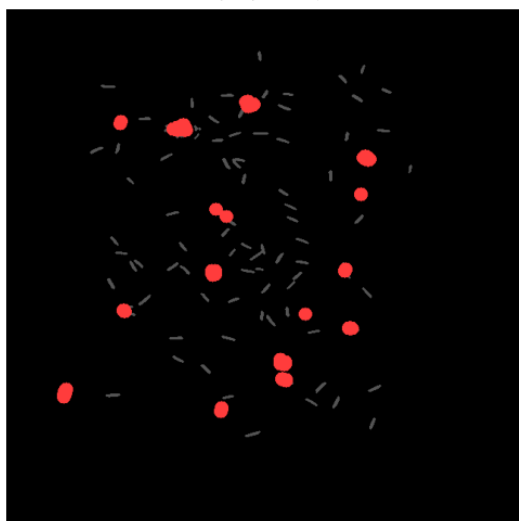
表 1: 三组数据上的总体结果, 误差以粒计, “最差”一列为最差单张误差。加粗为六种几何方法中每列最好的值, SAM 3 另列在最后作参照。B5 和 SAM 3 是在每个数据集上分别调到最好后的结果。

方法	D1 真实稀疏			D2 合成粘连			D3 低分辨率		
	MAE	准确率%	最差	MAE	准确率%	最差	MAE	准确率%	最差
B1 连通域计数	6.31	93.25	19	22.95	67.94	51	3.31	76.17	16
B2 面积估算	22.76	74.72	87	2.30	96.69	18	18.39	-30.64	76
B3 距离变换注水分割	74.90	17.77	123	8.20	88.63	24	8.27	42.82	18
B4 腐蚀标记注水分割 ^[2]	5.45	94.21	26	22.93	67.97	51	4.29	69.37	16
B5 凹点 + 椭圆拟合 ^[4]	5.76	—	—	2.08	—	—	3.32	—	—
本文方法	4.84	94.89	24	0.25	99.61	3	1.90	86.19	17
SAM 3 零样本	3.61	96.11	24	72.15	0.00	98	0.18	98.39	19

B4 在合成数据上的误差与 B1 逐档相同。一次 3×3 腐蚀不足以分开粘连块, 每块仍只产生一个种子, 注水分割什么也没切开, 等于退化成连通域计数。

B3 在真实照片上误差达 74.90 粒。它的阈值取自整张图距离变换的最大值, 而画面里的硬币远大于米粒, 把阈值整体抬高, 大多数米粒的种子被抹掉了。图 11 是同一张照片上两种取种子方式的对照, B3 全图只留下 19 个种子, 方法一按连通域分别标定, 得到 150 个。

B3 的全局阈值，只剩 19 个种子



本文的自适应种子，共 150 个

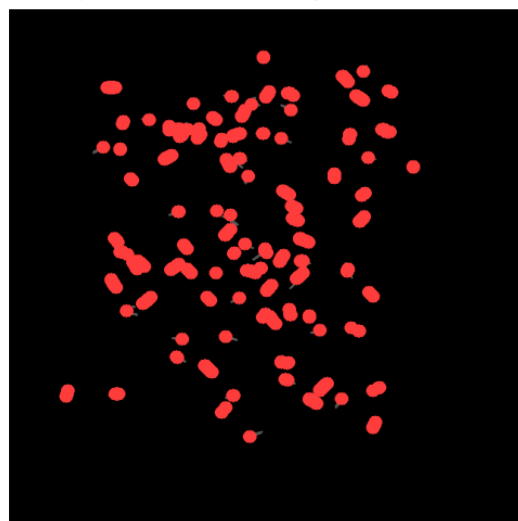


图 11: B3 的全局阈值与本文自适应种子的对照，红色为种子，灰色为二值图。本文对每个连通域分别放种子，粘连块会得到不止一个，因此总数多于米粒数。

B5 难以同时兼顾两种尺度。它在轮廓上作一个小圆找凹点，米粒只有 4 像素宽时，圆几乎盖住整粒米，孤立的单粒也会被判出凹点。把圆调大，它在 D2 上又几乎不再切分。需要说明，原文的椭圆拟合为穷举搜索，本文实现的是贪心近似，对它未必公平。

这三种方法失效的原因相同，都用了写死的数字。换一种米粒尺度，这些数字就不再合适，方法一的判据从图像自身量出，所以在三组数据上表现都比较稳。

4.4 误差随粘连程度的变化

再看粘连加重时方法一还准不准。结果见表 2 与图 12。粘连率自 0% 升至 80%，连通域计数的误差由 0.25 粒一路升到 38.88 粒，方法一在五档上均未超过 0.9 粒。图 13 是逐张的预测值与真值对照，格子集中在对角线附近，没有整体偏多或偏少。可见粘连加重时，方法一的误差只是缓慢上升，没有随粘连一起涨上去。

表 2: 合成数据上按粘连率分层的平均绝对误差，单位为粒，加粗为每一档误差最小的方法。

粘连率	B1	B2	B3	B4	B5	本文
0%	0.25	1.12	0.00	0.12	0.00	0.00
20%	12.50	1.12	4.50	12.50	7.62	0.12
40%	27.62	2.25	10.62	27.62	15.88	0.12
60%	35.50	2.50	11.12	35.50	16.38	0.12
80%	38.88	4.50	14.75	38.88	18.25	0.88

4.5 图像质量变差时的表现

这一小节找方法一的失效边界。本文在 D2 上人为施加模糊、噪声、对比度压缩与降采样四类退化，结果见表 3，其中模糊强度以单粒宽度 b_0 为单位。

噪声加到 45 个灰度级、对比度压到 8%，误差都没有上升。降采样到 1/6 时米粒只剩约 2 像素宽，中位误差为 4 粒，误差逐步上升而不是突然失效。模糊则不同。中位误差变化不大，平均误差却突然暴涨，原因是个别图像模糊后二值化整个选错，一张图数出上千粒。所以方法一怕的是模糊，噪声、低对比度和低分辨率它都扛得住。

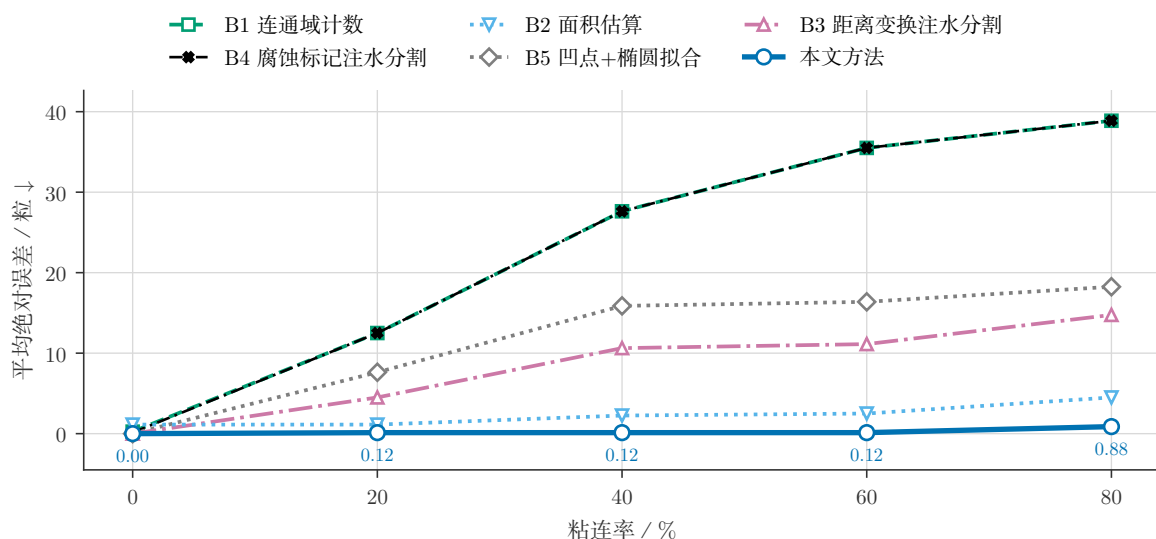


图 12: 计数误差随粘连率的变化。

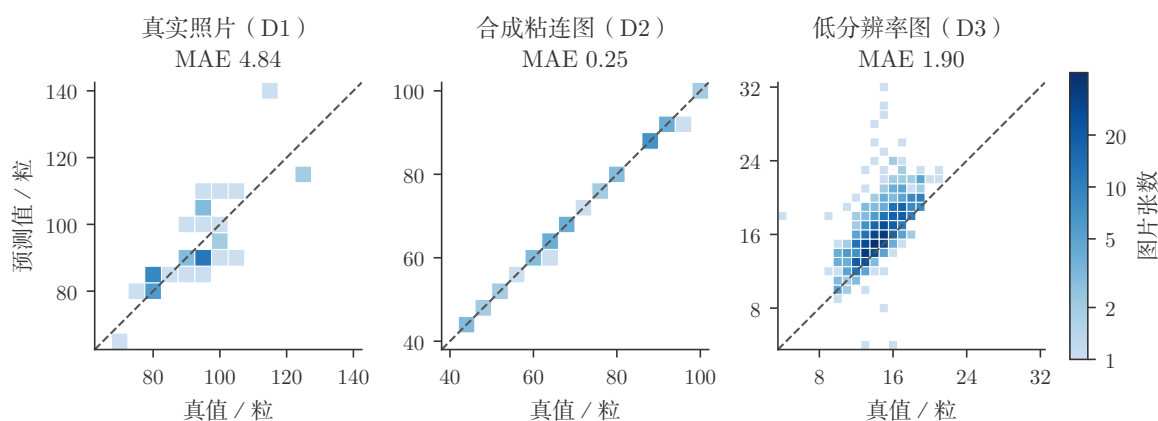


图 13: 预测值与真值对照的热力图。格子颜色表示落在该格的图片张数，虚线表示两者相等。

表 3: 合成粘连图 D2 在不同退化下的计数误差，共 40 张。“崩溃”指单张误差超过 100 粒，出现崩溃的格子加粗。

退化类型	强度	平均误差	中位误差	崩溃场景	B1 平均误差
高斯模糊	0.00 b_0	0.25	0.00	0/40	22.95
	0.20 b_0	251.30	0.50	2/40	156.95
	0.35 b_0	77.78	2.00	2/40	65.78
	0.70 b_0	73.92	15.50	13/40	59.95
加性噪声	0	0.25	0.00	0/40	22.95
	20	0.28	0.00	0/40	22.95
	45	0.23	0.00	0/40	22.98
对比度保留	8%	0.23	0.00	0/40	22.98
	30%	0.23	0.00	0/40	22.95
	100%	0.25	0.00	0/40	22.95
降采样倍率	1.0×	0.25	0.00	0/40	22.95
	3.0×	1.82	1.00	0/40	23.88
	6.0×	5.25	4.00	0/40	26.00

4.6 消融实验

这一小节逐项关掉或改动方法一的某一步，看每一步各起多大作用，结果见表 4。

表 4: 去掉或改动某一步后的平均绝对误差，单位为粒，第一行为完整方法。每列加粗的是误差最大的一项，即去掉后损失最大的那一步。

消融项	D1	D2	D3
完整方法	4.84	0.25	1.90
去掉尺度自标定, A_0 取数据集均值	8.20	0.17	1.85
去掉形状先验校正	5.31	1.12	1.58
去掉异物剔除	5.10	0.25	3.74
异物剔除去掉宽度判据	4.90	0.25	2.02
异物判据附加“更圆”条件，即最初写法	4.92	0.25	2.97
粘连判据改为“或”	6.00	0.07	1.54
种子碎块不合并	4.84	3.17	1.92
种子碎块不合并, β 取 0.45, 即修正前的写法	4.73	1.77	1.93
仅灰度通道	4.63	0.25	—
加 3×3 中值滤波	7.39	0.23	—

合并种子碎块管的是切分本身。不合并时，D2 的误差从 0.25 粒涨到 3.17 粒，一粒米常被切成两半。修正前的写法不合并碎块，只好把 β 抬到 0.45 来压住碎块，代价是许多粘连块切不开，D2 的误差为 1.77 粒。

形状先验校正，也就是按面积补数，兜底的是注水分割仍没切开的那些块。去掉它，D2 的误差从 0.25 粒涨到 1.12 粒，D1 从 4.84 粒涨到 5.31 粒。

异物剔除管的是 D3 的木框。去掉它，D3 的误差从 1.90 粒涨到 3.74 粒。

尺度自标定管的是真实照片里参差的米粒大小。改成整个数据集共用一个 A_0 后，D1 的误差从 4.84 粒涨到 8.20 粒。D2 反而从 0.25 粒降到 0.17 粒，因为合成图的米粒本来就大小整齐，一个固定值比每张图现量更稳。

只用灰度通道时 D1 稍好，为 4.63 粒，但 D3 离不开饱和度通道，所以候选通道仍都保留。

表中也有两项在个别数据上反而更好。粘连判据改成“或”后 D2、D3 的误差略降，去掉形状先验校正后 D3 也略降，但它们都让真实照片 D1 变差，所以没有采用。

这几步各管一类问题。去掉其中哪一步，都有一组数据明显变差。

4.7 与 SAM 3 对比

最后把方法一和视觉大模型 SAM 3^[15] 放在一起比。SAM 3 是去年冬天由 Meta 推出的视觉分割模型，由文本提示驱动，给出“rice grain”之类的提示，就能返回每粒米的掩膜，无需训练。本文先在 D1 上比较了不同提示词与阈值，结果见表 5，再把各提示词在三组数据上分别调到最好，结果见表 6。

表 5: SAM 3 在真实照片 D1 上使用不同提示词和阈值的平均绝对误差，单位为粒，加粗为误差最小的一格。

文本提示	$\tau = 0.15$	$\tau = 0.20$	$\tau = 0.25$	$\tau = 0.30$	$\tau = 0.35$	$\tau = 0.40$
grain	18.86	9.04	4.69	3.63	3.61	4.53
rice grain	8.71	6.41	8.12	13.45	28.41	51.78
white rice grain	18.10	10.00	7.16	5.92	5.88	6.71

需要如实说明，提示词选得恰当时，SAM 3 在 D1、D3 上比方法一更准，在重度粘连的 D2 上则是方法一更准。white seed 在三组数据上分别为 3.29、0.65、0.07 粒。但它对提示词相当敏感，五个提示词中有三个在合成图上一粒也没检出，其中包括看起来最自然的 rice grain，而模型不会提示自己失败。要知道哪个提示词可用，仍然得拿带标注的数据去试。

表 6: 各提示词在各数据集上分别调到最好时的误差, τ^* 为对应阈值, 加粗为每组数据上误差最小的提示词。

文本提示	D1		D2		D3	
	MAE	τ^*	MAE	τ^*	MAE	τ^*
rice grain	6.41	0.20	72.15	0.05	0.62	0.20
grain	3.61	0.35	72.15	0.05	0.07	0.35
white rice grain	5.88	0.35	72.12	0.05	0.08	0.20
white seed	3.29	0.40	0.65	0.40	0.07	0.35
seed	24.45	0.25	27.45	0.20	0.08	0.40

图 14 至图 16 是每组数据一个场景的逐粒结果, 便于核对。每粒米涂一种颜色, 相邻两粒颜色不同, 涂色半透明, 米粒本身的纹理仍看得见。虚线框与 $\times N$ 标出的区域并未被真正切开, 其粒数由面积推断而来。灰色并标 $\times$ 者为判定的异物, 不计入总数。

真实照片 (D1) 本文方法: 99 粒 (真值 98 粒) 真实照片 (D1) SAM 3: 93 粒 (真值 98 粒)



图 14: 真实照片 D1 上的结果, 左为本文方法、右为 SAM 3。

合成粘连图 (D2) 本文方法: 63 粒 (真值 64 粒) 合成粘连图 (D2) SAM 3: 64 粒 (真值 64 粒)

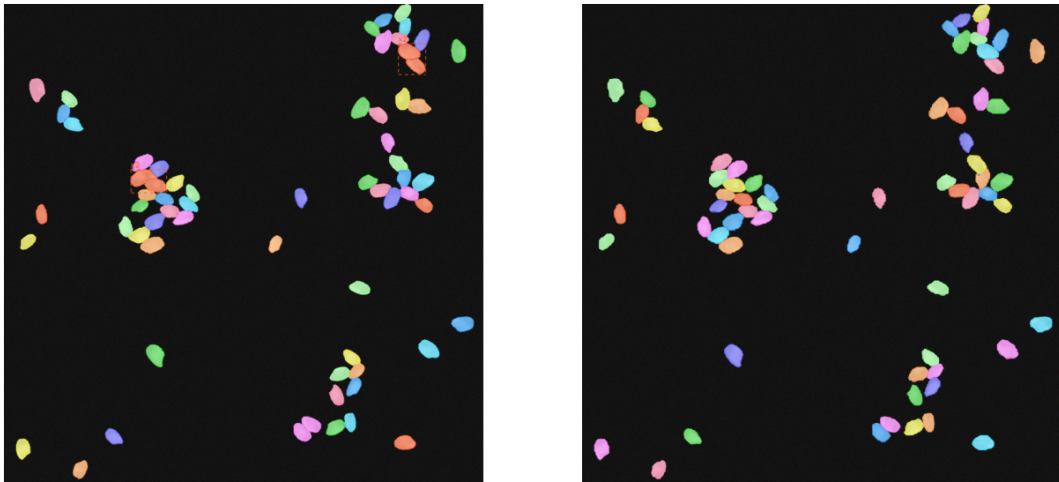


图 15: 合成粘连图 D2 中重度粘连场景的结果, 左为本文方法、右为 SAM 3。

D1 中 SAM 3 没有把硬币计入, 方法一却把硬币上的浮雕碎片数成了 8 粒, 详见附录 A。D2 中方法一数出 63 粒, 真值 64 粒, 只有两处是靠面积补齐的。D3 中木框亮边被正确剔除, 但漏掉了一粒颜色偏暗的米。

低分辨率图 (D3) 本文方法: 14 粒 (真值 15 粒) 低分辨率图 (D3) SAM 3: 15 粒 (真值 15 粒)

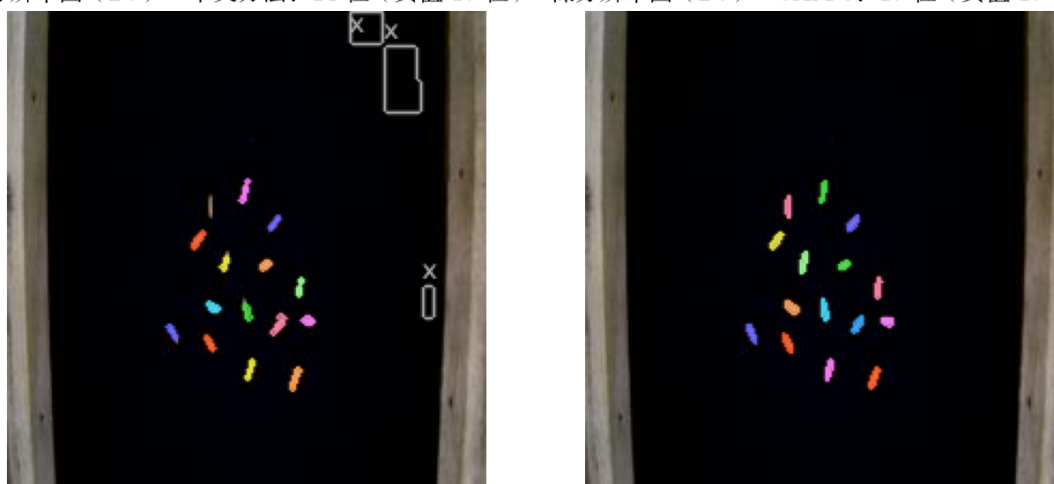


图 16: 低分辨率图 D3 上的结果, 左为本文方法、右为 SAM 3。

4.8 算力成本对比

准确度之外, 两种方法的算力开销差别很大。测量在同一台笔记本上进行, 配置为 Intel i9-13900H 处理器与 12 GB 显存的 RTX 4080 Laptop GPU。每组数据取 10 张图, 测每张的处理时间, 不含读图, 并记录峰值内存和显存, 结果见表 7。SAM 3 在 CPU 上太慢, 每组只测了 1 张。CPU 上用 32 位浮点, GPU 上用 16 位浮点 bfloat16。方法一的耗时是按第 6.1 节优化之后测的。

表 7: 单张图的处理时间和资源占用, 显存为分配峰值, SAM 3 在 CPU 上每组只测 1 张。每行加粗的是最省的一项。

项目	本文 CPU	本文 GPU	学生模型 GPU	SAM 3 GPU	SAM 3 CPU
D1 每张耗时 / s	0.37	0.15	0.014	0.21	17.7
D2 每张耗时 / s	0.63	0.40	0.025	0.21	17.3
D3 每张耗时 / s	0.033	0.071	0.002	0.21	18.3
模型参数量	0	0	0.27 M	840 M	840 M
权重文件 / MB	0	0	1	3281	3281
模型加载 / s	—	—	0.1	3.1	2.5
峰值内存 / GB	0.30	0.70	1.17	5.61	5.07
峰值显存 / GB	—	0.10	0.23	2.50	—

有显卡时 SAM 3 每张约 0.21 s, 方法一的 GPU 版在 D1 上更快, 为 0.15 s, 在 D2 上略慢。但 SAM 3 需先加载 8.4 亿参数、3.2 GB 权重, 运行时占用 5.6 GB 内存与 2.5 GB 显存, 方法一没有需要学习的参数, GPU 版只占 0.09 GB 显存。没有显卡时差距更大。SAM 3 每张耗时 17 s 以上, 方法一在 CPU 上为 0.03 至 0.63 s。也就是说, 有显卡时两者速度相近, 没有显卡时方法一快得多, 而且几乎不占内存。

4.9 本节小结

方法一在三组数据上都比五种常见做法准, 粘连加重时误差也几乎不上升。它的弱点有两处, 一是怕模糊, 二是在低分辨率的 D3 上与 SAM 3 差距明显。SAM 3 在真实照片和低分辨率图上更准, 却要挑提示词、要显卡。能不能兼得两者的长处, 是第 5 节方法二要回答的问题。

5 方法二 SAM3 作为教师模型训练专用小模型

5.1 方法二思路的由来

第二种方法是做完上一节的实验之后才想到的。那里留下一个我不太满意的结果。本文方法在 D3 上无论怎么调都与 SAM 3 差距明显。失败的样子还大多属于同一类，木框亮边被当成米，颜色偏暗的米被漏掉。追究下去，几何判据只知道米长什么样，并不知道什么是米，这一点在第 6.3 节还会谈到。既然 SAM 3 知道，能否把它知道的东西教给一个小模型？

5.2 具体实现方式

标签怎么来。用 SAM 3 处理每张照片，取其每个掩膜的质心，得到一串点坐标作为伪标签。合成图 D2 则直接采用生成时记录的米粒中心，不必经过教师。把大模型的输出当作参考答案，是核对过之后才敢用的。拿它与人工标注逐点匹配，教师给出的点在 D3 上准确率 0.993、召回率 1.000、计数误差 0.10 粒，在 D1 上依次为 0.957、0.967 与 3.29 粒。这两组数据上的伪标签足够干净。人工标注全程不参与训练，只用于测试。学生的成绩因此仍是拿人工标注量出来的。图 17 是其中一张照片的逐点对照，漏掉的和多给的都是少数。

人工标注 116 粒，SAM 3 给出 114 粒
配对上 107 粒，漏 9 粒，多 7 点



局部放大
橙圈为漏掉的米粒，蓝方块为多给的点

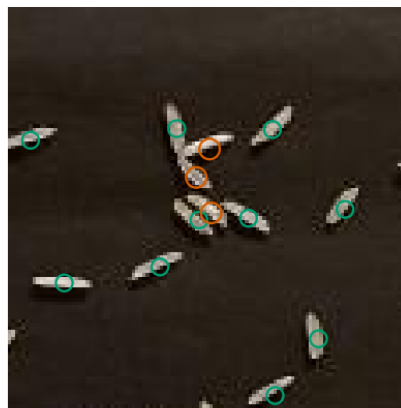


图 17: SAM 3 给出的点与人工标注的对照。绿圈为两者配对上的米粒，橙圈为 SAM 3 漏掉的，蓝方块为它在同一粒米上多给的点。

学生学什么。学的不是分割，而是密度图，即在每个米粒处放一个高斯，整张图的积分即为粒数。这样安排有两点考虑。其一，米粒只有 4 像素宽，要求网络给出清晰的实例边界并不现实。让它在米粒处“鼓起一个包”却是可行的。其二，损失函数不依赖于把相邻两粒分开，而那恰是几何方法最吃力之处。高斯的宽度按每张图自身的米粒间距确定，因此 4 像素与 25 像素的米粒在网络看来尺度相当。从教师的掩膜到学生的预测，整条流程见图 18。

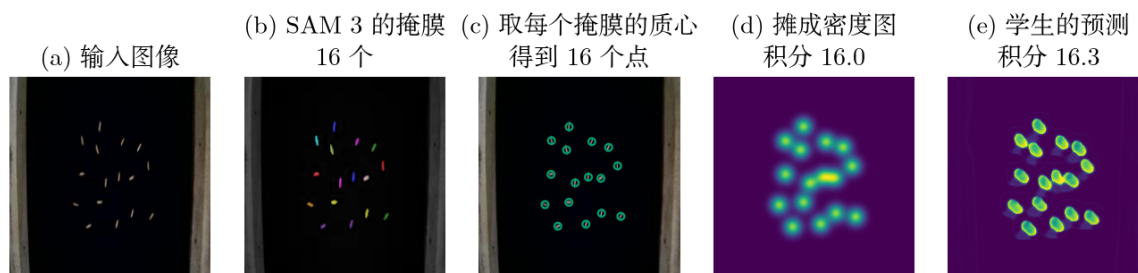


图 18: 方法二的流程。教师给出掩膜，取质心得到点标签，每个点摊成一个高斯就是训练目标，学生学着把这张密度图预测出来，其积分即为粒数。

网络与训练。三层 U-Net，27 万参数，权重文件 1 MB。按 7:1:2 划分训练、验证与测试集，划分以源图为单位，以免同一张照片的增强副本跨集泄漏。在 RTX 4080 上训练 6000 步，约 6 分钟。

表 8: 学生模型与本文方法、教师在同一批测试图上的平均绝对误差，单位为粒。真值在 D1、D3 上为人工标注，在 D2 上为合成真值，三者都未用于学生的训练。

数据集	张数	直接数连通域	方法一	方法二学生模型	SAM 3 教师
D1 真实照片	10	6.20	4.60	3.91	2.40
D2 合成粘连	8	24.38	0.25	1.29	1.00
D3 低分辨率	144	3.23	1.76	0.42	0.06
合计	162	4.46	1.86	0.68	0.25

5.3 结果与代价

结果见表 8。为便于比较，表中一并列出了不作任何粘连处理时直接统计连通域的结果 4.46 粒，四者测的是同一批图。学生模型的总体误差为 0.68 粒，优于方法一的 1.86 粒。在训练数据最多的 D3 上差距最明显，为 0.42 对 1.76 粒。D1 上亦更好，为 3.91 对 4.60 粒。D2 上则是方法一更好，为 0.25 对 1.29 粒。它仍不及教师的 0.25 粒，这符合预期，蒸馏一般不会超过教师。但它只有 27 万参数、1 MB 权重，约为 SAM 3 的三千分之一。每张图耗时 2 至 25 ms，比 SAM 3 快 8 到 100 倍，见表 7。更要紧的是不必挑选提示词。SAM 3 换一个提示词就可能一粒也检不出，学生则把选定的那套行为固化了下来。

图 19 是 D3 上的一个例子，也正是第 5 节开头那类失败。画面左侧的木框亮边被方法一当成一处粘连块，按面积折算成 8 粒，全图数出 24 粒，真值为 14 粒。学生模型的密度图上，木框那一带没有鼓起任何包，积分得 14.2 粒。它没有用到更多的形状判据，只是从教师那里学到了米粒该是什么样子。



图 19: 同一张低分辨率图上两种方法的对照。中图里左侧的虚线框是方法一把木框亮边当作粘连块后按面积折算的 8 粒，右图为学生模型预测的密度图。

6 讨论

6.1 让程序跑得更快

这一小节记录方法一从每张 6.6 s 提速到 0.63 s 的过程，结论是收益最大的几步都不需要显卡。

最初 D2 每张需 6.6 s，我的第一反应是“慢就上 GPU”。先做了一次时间统计，看程序各步骤分别花掉多少时间，结果与设想相去甚远。Otsu、连通域标记、距离变换这些逐像素运算，8

个候选全算一遍也只有约 50 ms，不到总时间的 1%。**98% 花在逐个连通域算凸包和长短轴上**，那是 Python 里的逐对象循环，搬到 GPU 上并不会变快。

能想到这一点，和上学期的两门课有直接关系。**操作系统原理**讲进程与线程调度时说过，线程不是越多越好。线程数超过核数之后，时间会花在上下文切换和争抢上，反而更慢。所以当我看到“一张图占了 17 个核却还很慢”时，第一反应是去查线程数，而不是去买更好的显卡。**计算机组成原理**讲的则是另一半。先看程序把时间花在哪一类操作上，再决定优化什么。并行部件也都有启动开销，粒度太小时并行反而不划算。

第一步，不算用不到的东西。面积和外接框用 OpenCV 的连通域统计一次取回，长短轴改用二阶矩批量算出，凸包只给面积大于 $0.3A_0$ 的区域算。D2 由此降到 2.7 s。

第二步，把线程数限制为 1。我本以为程序是单线程的，实测却发现处理一张图占满了 17 个核。标定中有大量很小的矩阵运算，OpenBLAS 默认开 20 个线程去应付，调度开销远大于计算本身。锁成单线程后，单张反而更快，D1 由 693 ms 降至 431 ms，D3 由 82 ms 降至 45 ms。多进程起初几乎没有加速，也是因为各进程争抢同一批核。线程锁定之后再按图并行，D3 用 20 个进程处理 240 张仅需 2.0 s。

第三步，调整判断顺序。这是收益最大的一步，改动却只是把代码的次序换了一下。候选的得分 $NA_0\gamma$ 不含凸实度，用到凸实度的只有“这批区域像不像米”那道合理性检查。原先先检查、再排序，每个候选都得先把凸包算出来。改为先按得分排序、再自高而低逐个检查后，选出的候选不变，排在其后的候选一次凸包也不必算。D2 由 2.7 s 降至 0.63 s，D1 降至 0.37 s，D3 降至 0.033 s。以上三步在三组共 810 张图上逐张比对，计数与优化前一致。

第四步，把逐像素的部分放到 GPU 上。通道转换、Otsu、开闭运算、连通域标记与距离变换改用 CuPy 和 cuCIM，注水分割仍留在 CPU 上。GPU 版在 D1、D2 上分别为 0.15 s 和 0.40 s，但在 D3 上反而比 CPU 慢一倍，为 0.07 s 对 0.03 s。D3 的图只有 224×224 ，GPU 启动核函数的开销比算完整张图还多。GPU 版的计数与 CPU 版接近，但并不逐张一致，正文结果仍以 CPU 版为准。

这样看来，收益最大的三步都不需要显卡，最先想到的 GPU 反倒是收益最小、代价最大的一步。先测量、再优化，这方法是我这次最实在的收获。

6.2 尝试过但效果不好的办法

上采样。真实照片中的米粒只有 4 至 6 像素宽，我曾设想先把粘连块放大再算距离变换。实测未见改善——插值并不会凭空增加原图中不存在的信息。相关代码已删除。

中值滤波。为压住饱和度和通道的噪点，我曾加入 3×3 中值滤波，当时确有效果。但式 (1) 补上 γ 之后，噪点已被排除，两者功能重复，而中值滤波还会把本就只有几像素宽的米粒磨掉一圈。去掉它后 D1 误差由 7.39 粒降至 4.84 粒。由此得到的经验，是加入新机制之后应当回头检查旧的补丁是否已成多余。

按 D3 调碎屑门限。调高碎屑门限能让 D3 明显变好，D1 却随之变差。查下去才发现，D1 中那些“又小又圆”的区域其实是真米粒——4 像素尺度下被阈值削去一截的米粒本就长这样。这等于针对单个数据集调参，故未采用。

6.3 一些不足

下面每一条先说看到的现象，再说原因。

画面里有比米更整齐的东西时，会把它当成米。我另找过一组带直尺的照片^[16]，想补充粘连数据。结果在不少图上，程序选出的前景是直尺的刻度，而不是米粒，见图 20。原因在于式 (1) 挑二值化方案时，看的是画面里有多少块大小一致的小东西。刻度线大小整齐、对比强烈，比颜色浅、大小不一的米粒更符合这个标准。换句话说，方法一只认形状，不认得什么是米。这组照片里还有不少是几个品种的米放在一起对比，粒径相差很大，不符合“同一张图里米粒大小相近”这一前提，所以最终没有用作测试集。

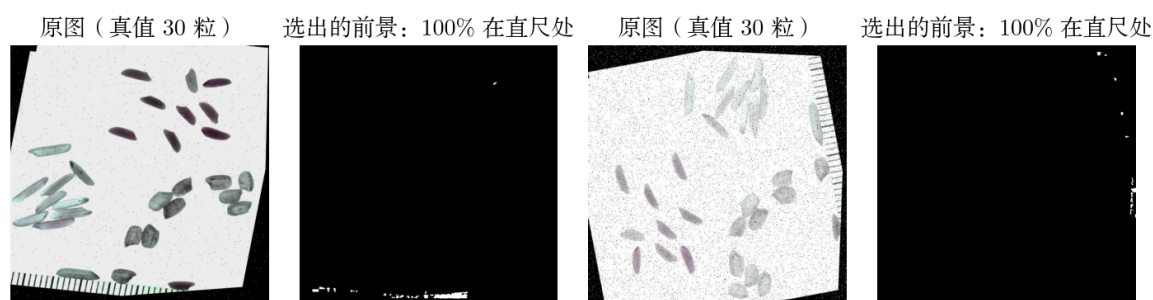


图 20: 方法被直尺刻度误导的两个例子，每例左为原图、右为选出的前景。

图像模糊时，可能选错二值化方案。第 4.5 节里出现崩溃的几张图，都是模糊造成的。模糊之后，饱和度通道被阈值切成大量小碎点。碎点数目多，式 (1) 的得分反而略高于正确的方案，程序便选错了，一张图数出上千粒。这个问题目前还没有解决。

硬币有浮雕时，可能被数成几粒米。剔除硬币的判据要求硬币先被二值化成完整的一块。硬币表面有浮雕时，二值化后它会碎成几块，每块都和米粒差不多大，于是被当成米计了数。附录 A 中就有这样一例。

D3 上平均每张多数约 1 粒。多出来的主要是木框边缘的亮块被当成了米，而不是把一粒米切成了两粒。

合成数据上的成绩偏乐观。D2 背景干净、米粒大小一致、光照均匀，条件比真实照片好得多。连“总面积除以单粒面积”这样简单的办法，在 D2 上也只差 2.30 粒，在真实照片上却差 22.76 粒。所以 D2 上的数字只说明方法能处理粘连，不代表真实照片上也能达到同样的精度。

7 结论

本文实现了一种基于注水分割的粘连米粒计数方法。它先由图像自身量出单粒米的尺度，据此判断粘连、控制种子数量，最后按面积与形状复核切分结果。三组数据上的平均绝对误差分别为 4.84、0.25 与 1.90 粒，优于五种对比方法。

做完这份作业，体会最深的一点是，这个问题没有想象中这么简单，我看到问题的第一眼，认为只要对二值化图像稍加处理即可。但遇到了挫折，于是我去网上搜集论文，看看有没有好的方法可以参考。受到了分水岭算法的启发，针对这个问题具体化，于是有了方法一。对于方法二，是一种偷懒的黑箱做法，效果好但不可解释。我做大创项目时使用过强大的 SAM3 模型，于是有了蒸馏学生模型的想法。

我将在文末附上更多的效果图和参考文献，头一次用 Latex 写报告，不太熟练，文中若有不恰当之处，还请老师批评指正。

A 更多计数结果

每组数据按顺序等间隔取两个场景，而非择优挑选，因此呈现的是方法的一般表现。左为本文方法、右为 SAM 3，画法与正文图 14 相同。

A.1 真实照片 D1

图 21 与图 22 两个场景中，本文方法分别数出 99 粒与 82 粒，真值为 98 粒与 82 粒。SAM 3 在两个场景中都没有把硬币计入，它本就知道硬币不是米。本文方法在第二个场景里靠第 3.6 节的异物判据把硬币整块剔除，第一个场景却没有剔除成功。那枚硬币上的浮雕纹路被二值化成六块碎片，没有连成一整块，每块的大小又都接近米粒，异物判据认不出它们，结果按面积算作 8 粒米。其余区域比真值少数 7 粒，两者相抵，总数才只差 1 粒。这一处按涂色画出来一眼就能看到，只看总数却看不出来。

本文方法：99 粒（真值 98 粒）



SAM 3：93 粒（真值 98 粒）



图 21: 真实照片 D1 场景一，左为本文方法、右为 SAM 3。

本文方法：82 粒（真值 82 粒）



SAM 3：84 粒（真值 82 粒）



图 22: 真实照片 D1 场景二，左为本文方法、右为 SAM 3。

A.2 合成粘连图 D2

图 23 的场景没有粘连，两种方法都数出 91 粒，和真值一致。第二个场景粘连率 40%，真值 93 粒，本文方法 93 粒、SAM 3 94 粒。图 24 中只有两处虚线框是按面积补出粒数的，其余粘连块都被真正切开了。

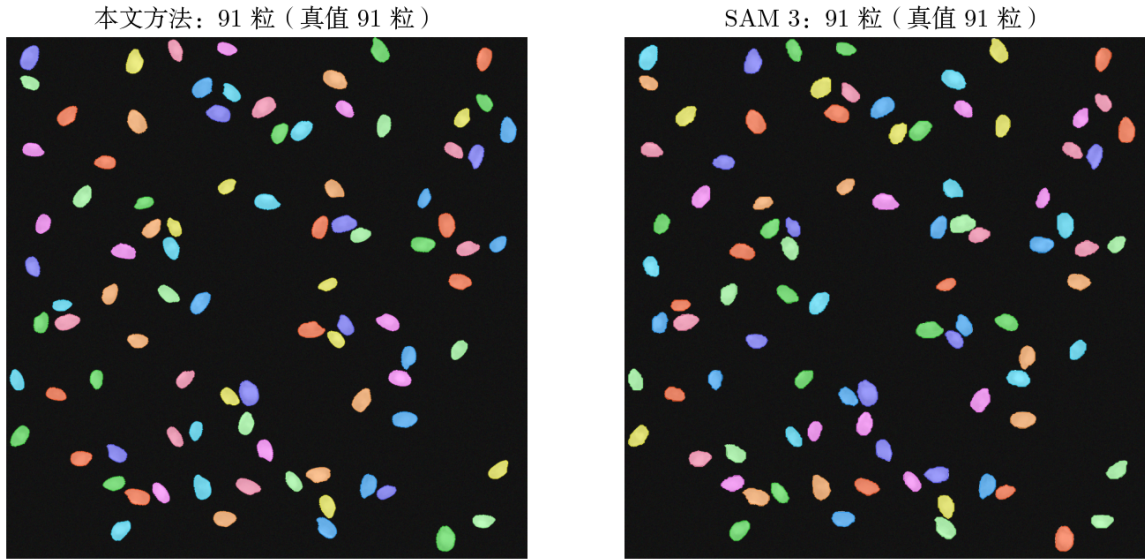


图 23: 合成粘连图 D2 中不粘连的场景，左为本文方法、右为 SAM 3。

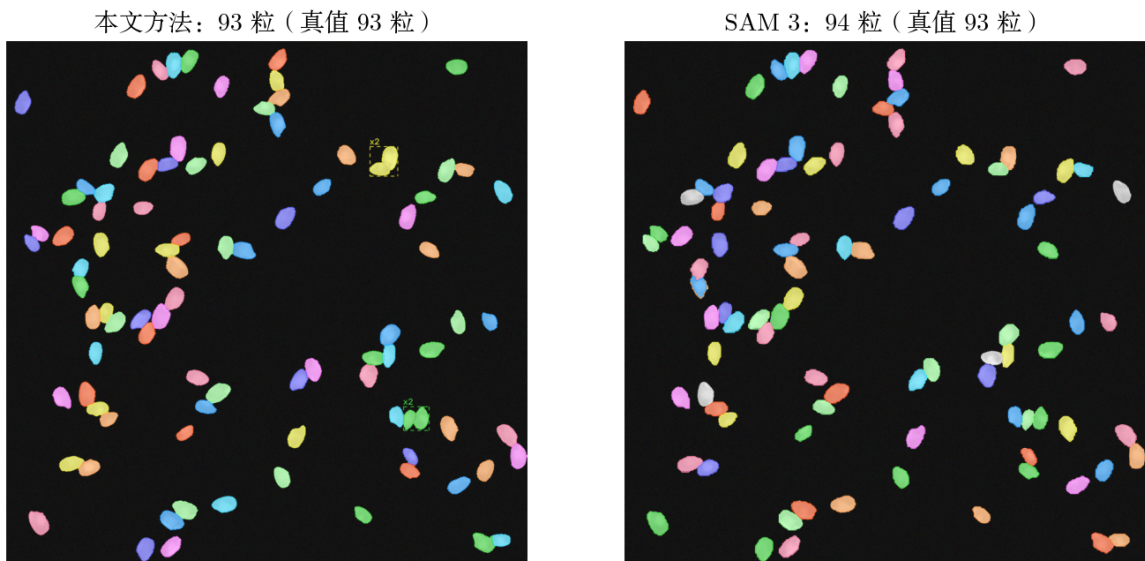


图 24: 合成粘连图 D2 中粘连率 40% 的场景，左为本文方法、右为 SAM 3。

A.3 低分辨率图 D3

图 25 与图 26 两个场景真值都是 15 粒，本文方法分别数出 14 粒与 16 粒。SAM 3 的轮廓则几乎贴合每一粒米。少数的那一粒颜色偏暗，二值化时没分出来。多数的那一个来自本框边缘的亮块。米粒只有 4 像素宽时，形状所能提供的信息已经很少，这也是本文方法与 SAM 3 差距最大的一组。

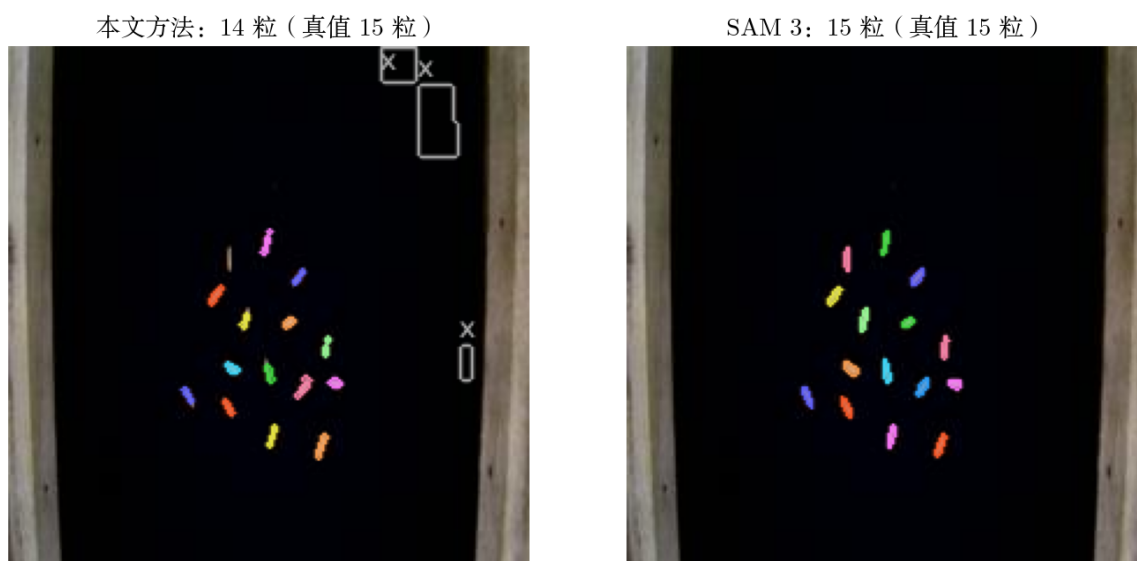


图 25: 低分辨率图 D3 场景一，左为本文方法、右为 SAM 3。

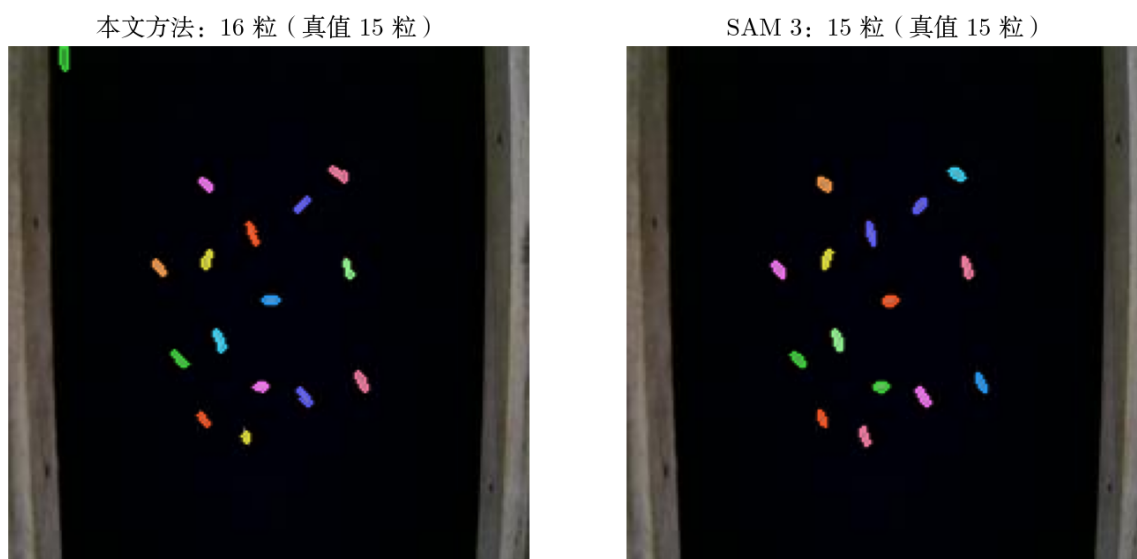


图 26: 低分辨率图 D3 场景二，左为本文方法、右为 SAM 3。

B 代码、数据与复现方式

代码、数据来源说明与复现步骤都放在 <https://github.com/stars-spark/asw-spc-rice-counting> 欢迎老师检查！D1、D3 与 D2 所用的单粒米素材取自 Roboflow Universe 上的公开数据集。

参考文献

- [1] Otsu N. A threshold selection method from gray-level histograms[J]. IEEE Transactions on Systems, Man, and Cybernetics, 1979, 9 (1): 62-66.
- [2] Kurade C, Meenu M, Kalra S, et al. An automated image processing module for quality evaluation of milled rice[J/OL]. Foods, 2023, 12 (6): 1273. DOI:10.3390/foods12061273.
- [3] Tan Suiyan, Ma Xu, Mai Zhijie, et al. Segmentation and counting algorithm for touching hybrid rice grains[J/OL]. Computers and Electronics in Agriculture, 2019, 162: 493-504. DOI:10.1016/j.compag.2019.04.030.
- [4] Avzalov D R, Komyshev E G, Afonnikov D A. Counting touching wheat grains in images based on elliptical approximation[J/OL]. Vavilovskii Zhurnal Genetiki i Seleksii, 2025, 29 (4): 608-614. DOI:10.18699/vjgb-25-64.
- [5] Vincent L, Soille P. Watersheds in digital spaces: an efficient algorithm based on immersion simulations [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1991, 13 (6): 583-598.
- [6] Gamarra M, Zurek E, Escalante H J, et al. Split and merge watershed: a two-step method for cell segmentation in fluorescence microscopy images[J/OL]. Biomedical Signal Processing and Control, 2019, 53: 101575. DOI:10.1016/j.bspc.2019.101575.
- [7] 凌云, 王一鸣, 孙明, 等. 基于流域算法的谷物籽粒图像分割技术[J]. 农业机械学报, 2005, 36 (3): 95-98.
- [8] 杨蜀秦, 何东健, 宁纪锋. 一种基于分水岭的连接米粒自动分割算法[J]. 光子学报, 2009, 38 (10): 2728-2732.
- [9] 荀一, 鲍官军, 杨庆华, 等. 粘连玉米籽粒图像的自动分割方法[J]. 农业机械学报, 2010, 41 (4): 163-167.
- [10] 柳冠伊, 刘平义, 魏文军, 等. 玉米果穗粘连籽粒图像分割方法[J/OL]. 农业机械学报, 2014, 45 (9): 285-290. DOI:10.6041/j.issn.1000-1298.2014.09.046.
- [11] 李文勇, 李明, 钱建平, 等. 基于形状因子和分割点定位的粘连害虫图像分割方法[J/OL]. 农业工程学报, 2015, 31 (5): 175-180. DOI:10.3969/j.issn.1002-6819.2015.05.025.
- [12] annotation-gnc0j. rice (v1) [DS/OL]. Roboflow Universe (2024) [2025-01-09]. <https://universe.roboflow.com/annotation-gnc0j/rice-nqqbr>.
- [13] thanaree. RICE (v2) [DS/OL]. Roboflow Universe (2023) [2023-12-16]. <https://universe.roboflow.com/thanaree/rice-fgvkm>.
- [14] ezeziel-se47x. RICE (v3) [DS/OL]. Roboflow Universe (2024) [2024-11-17]. <https://universe.roboflow.com/ezeziel-se47x/rice-ljhi4>.
- [15] Meta AI. SAM 3: segment anything with concepts[EB/OL]. (2025) [2026-09-18]. <https://huggingface.co/facebook/sam3>.
- [16] asia-pacific-university-xxaah. rice count (v2) [DS/OL]. Roboflow Universe (2023) [2024-07-12]. <https://universe.roboflow.com/asia-pacific-university-xxaah/rice-count>.